

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool: Debugging & Optimisation Task (Low)
Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Identify and correct errors in a given SQL CREATE TABLE statement with incorrect syntax and missing constraints.	BL2 – Understand
2	Debug an SQL query that incorrectly uses aggregate functions without GROUP BY and fix the issue. Bloom's Level:	BL2 – Understand
3	Correct a faulty SELECT query where column names are misspelled or improperly referenced.	BL1 – Remember
4	Fix errors in an SQL query using JOIN, where the join condition is missing or incorrect.	BL3 – Apply
5	Identify and correct a subquery error where multiple values are returned instead of a single value.	BL2 – Understand
6	Debug a VIEW creation statement that contains invalid column references or syntax errors.	BL1 – Remember
7	Examine an SQL query with unnecessary conditions and optimize it by simplifying the WHERE clause.	BL3 – Apply
8	Identify violations of integrity constraints in a given table and correct them.	BL2 – Understand

9	Fix a relation that violates 1NF or 2NF by removing repeating groups or partial dependencies.	BL3 – Apply
10	Identify redundant attributes in a relation and apply basic normalization (up to 3NF) to improve design.	BL3 – Apply

Code of Conduct:

I **SANTHOSH.E (192511470)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

E. SANTHOSH

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Q1. Identify and correct errors in a given SQL CREATE TABLE statement with incorrect syntax and missing constraints.

✗ Incorrect SQL

```
mysql> CREATE TABLE Student
  -> Student_ID INT,
  -> Student_Name VARCHAR(30),
  -> Age INT
  -> );
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'Student_ID INT,
Student_Name VARCHAR(30),
Age INT
)' at line 2
```

🔍 Errors

- Missing opening bracket (.).
- No PRIMARY KEY.
- Incorrect CREATE TABLE syntax.

✓ Correct SQL

```
mysql> use AT3;
Database changed
mysql> CREATE TABLE Student(
  -> Student_ID INT PRIMARY KEY,
  -> Student_Name VARCHAR(30) NOT NULL,
  -> Age INT
  -> );
Query OK, 0 rows affected (0.04 sec)
```

OUTPUT

```
mysql> DESC Student;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| Student_ID | int           | NO   | PRI | NULL    |       |
| Student_Name | varchar(30)   | NO   |     | NULL    |       |
| Age        | int           | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.04 sec)
```

Justification

The corrected statement adds proper table syntax, a primary key, and a NOT NULL constraint to ensure valid table creation and data integrity.

2) Debug an SQL query that incorrectly uses aggregate functions without GROUP BY.
(BL2 – Understand)

Create table and Insert data

```
mysql> CREATE TABLE Student(
  -> Student_ID INT PRIMARY KEY,
  -> Student_Name VARCHAR(30),
  -> Dept_ID VARCHAR(5),
  -> Marks INT
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Student VALUES
  -> (101, 'Rahul', 'D101', 85),
  -> (102, 'Priya', 'D101', 75),
  -> (103, 'Kiran', 'D102', 90),
  -> (104, 'Sneha', 'D102', 80);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

✗ Incorrect SQL

```
mysql> SELECT Dept_ID,  
-> AVG(Marks)  
-> FROM Student;  
ERROR 1140 (42000): In aggregated query without GROUP BY, expression #1  
of SELECT list contains nonaggregated column 'at3.Student.Dept_ID'; this  
is incompatible with sql_mode=only_full_group_by
```

🔍 Error

Aggregate function is used with a normal column without GROUP BY.

✓ Correct SQL

```
mysql> SELECT Dept_ID,  
-> AVG(Marks) AS Average_Marks  
-> FROM Student  
-> GROUP BY Dept_ID;
```

Output

```
mysql> SELECT Dept_ID,  
-> AVG(Marks) AS Average_Marks  
-> FROM Student  
-> GROUP BY Dept_ID;  
+-----+-----+  
| Dept_ID | Average_Marks |  
+-----+-----+  
| D101    | 80.0000       |  
| D102    | 85.0000       |  
+-----+-----+  
2 rows in set (0.01 sec)
```

Justification

GROUP BY groups rows based on department so that AVG() is calculated separately for each department.

Q3. Correct a faulty SELECT query where column names are misspelled.
(BL1 – Remember)

Create table and Insert data

```
mysql> CREATE TABLE Student(  
  -> Student_ID INT PRIMARY KEY,  
  -> Student_Name VARCHAR(30),  
  -> Marks INT  
  -> );  
Query OK, 0 rows affected (0.02 sec)  
  
mysql> INSERT INTO Student VALUES  
  -> (101, 'Rahul', 85),  
  -> (102, 'Priya', 90),  
  -> (103, 'Kiran', 75);  
Query OK, 3 rows affected (0.01 sec)  
Records: 3  Duplicates: 0  Warnings: 0
```

✗ Incorrect SQL

```
mysql> SELECT Studnt_Name,  
  -> Mrks  
  -> FROM Student;  
ERROR 1054 (42S22): Unknown column 'Studnt_Name' in 'field list'
```

🔍 Errors

- Studnt_Name → Incorrect spelling.
- Mrks → Incorrect spelling.

✓ Correct SQL

```
mysql> SELECT Student_Name,  
  -> Marks  
  -> FROM Student;  
  
+-----+-----+  
| Student_Name | Marks |  
+-----+-----+  
| Rahul       | 85    |  
| Priya       | 90    |  
| Kiran       | 75    |  
+-----+-----+  
3 rows in set (0.00 sec)  
  
mysql> |
```

Output

Student_Name	Marks
Rahul	85
Priya	90
Kiran	75

Justification

Column names must exactly match those defined in the table schema. Correct spelling ensures successful query execution.

Q4. Fix errors in an SQL query using JOIN, where the join condition is missing or incorrect.

(BL3 – Apply)

Create table and Insert data

```
mysql> CREATE TABLE Student(  
  -> Student_ID INT PRIMARY KEY,  
  -> Student_Name VARCHAR(30),  
  -> Dept_ID VARCHAR(5)  
  -> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql>  
mysql> CREATE TABLE Department(  
  -> Dept_ID VARCHAR(5) PRIMARY KEY,  
  -> Dept_Name VARCHAR(30)  
  -> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> INSERT INTO Student VALUES  
  -> (101, 'Rahul', 'D101'),  
  -> (102, 'Priya', 'D102'),  
  -> (103, 'Kiran', 'D101');  
Query OK, 3 rows affected (0.01 sec)  
Records: 3 Duplicates: 0 Warnings: 0  
  
mysql>  
mysql> INSERT INTO Department VALUES  
  -> ('D101', 'CSE'),  
  -> ('D102', 'ECE');  
Query OK, 2 rows affected (0.01 sec)  
Records: 2 Duplicates: 0 Warnings: 0
```

✗ Incorrect SQL

```
mysql> SELECT Student_Name,  
-> Dept_Name  
-> FROM Student  
-> JOIN Department;  
+-----+-----+  
| Student_Name | Dept_Name |  
+-----+-----+  
| Rahul        | ECE      |  
| Rahul        | CSE      |  
| Priya        | ECE      |  
| Priya        | CSE      |  
| Kiran        | ECE      |  
| Kiran        | CSE      |  
+-----+-----+  
6 rows in set (0.00 sec)
```

🔍 Error

JOIN condition (ON) is missing.

✓ Correct SQL

```
mysql> SELECT S.Student_Name,  
-> D.Dept_Name  
-> FROM Student S  
-> JOIN Department D  
-> ON S.Dept_ID=D.Dept_ID;  
+-----+-----+  
| Student_Name | Dept_Name |  
+-----+-----+  
| Rahul        | CSE      |  
| Priya        | ECE      |  
| Kiran        | CSE      |  
+-----+-----+  
3 rows in set (0.00 sec)  
  
mysql> |
```

Output

Student_Name	Dept_Name
Rahul	CSE
Priya	ECE
Kiran	CSE

Justification

The ON clause specifies how the two tables are related. Without it, the query may produce a Cartesian product or fail.

Q5. Identify and correct a subquery error where multiple values are returned instead of a single value.

(BL2 – Understand)

Create table and Insert data

```
mysql> CREATE TABLE Student(  
-> Student_ID INT PRIMARY KEY,  
-> Student_Name VARCHAR(30),  
-> Dept_ID VARCHAR(5)  
-> );  
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> CREATE TABLE Department(  
-> Dept_ID VARCHAR(5) PRIMARY KEY,  
-> Dept_Name VARCHAR(30)  
-> );  
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> INSERT INTO Student VALUES  
-> (101, 'Rahul', 'D101'),  
-> (102, 'Priya', 'D102'),  
-> (103, 'Kiran', 'D101');  
Query OK, 3 rows affected (0.00 sec)  
Records: 3 Duplicates: 0 Warnings: 0
```

```
mysql>  
mysql> INSERT INTO Department VALUES  
-> ('D101', 'CSE'),  
-> ('D102', 'ECE');  
Query OK, 2 rows affected (0.01 sec)  
Records: 2 Duplicates: 0 Warnings: 0
```

✗ Incorrect SQL

```
mysql> SELECT Student_Name
-> FROM Student
-> WHERE Dept_ID =
-> (
-> SELECT Dept_ID
-> FROM Department
-> );
ERROR 1242 (21000): Subquery returns more than 1 row
```

🔍 Error

The subquery returns multiple department IDs, but the = operator expects only one value.

✓ Correct SQL

```
mysql> SELECT Student_Name
-> FROM Student
-> WHERE Dept_ID IN
-> (
-> SELECT Dept_ID
-> FROM Department
-> );
+-----+
| Student_Name |
+-----+
| Rahul        |
| Priya        |
| Kiran        |
+-----+
3 rows in set (0.00 sec)
```

Output

Student_Name
Rahul
Priya
Kiran

Justification

The IN operator is used when the subquery returns multiple values. It compares the column against all returned values, preventing the "subquery returns more than one row" error.

Q6. Debug a VIEW creation statement that contains invalid column references or syntax errors
(BL1 – Remember)

Create Table and Insert Data

```
mysql> CREATE TABLE Student(  
    -> Student_ID INT PRIMARY KEY,  
    -> Student_Name VARCHAR(30),  
    -> Marks INT  
    -> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> INSERT INTO Student VALUES  
    -> (101, 'Rahul', 85),  
    -> (102, 'Priya', 90),  
    -> (103, 'Kiran', 75);  
Query OK, 3 rows affected (0.01 sec)  
Records: 3  Duplicates: 0  Warnings: 0
```

✗ Incorrect SQL

```
mysql> CREATE VIEW Student_View AS  
    -> SELECT Name, Marks  
    -> FROM Student;  
ERROR 1054 (42S22): Unknown column 'Name' in 'field list'
```

✓ Correct SQL

```
mysql> CREATE VIEW Student_View AS  
-> SELECT Student_Name, Marks  
-> FROM Student;  
Query OK, 0 rows affected (0.01 sec)
```

OUTPUT

```
mysql> SELECT * FROM Student_View;  
+-----+-----+  
| Student_Name | Marks |  
+-----+-----+  
| Rahul       | 85    |  
| Priya       | 90    |  
| Kiran       | 75    |  
+-----+-----+  
3 rows in set (0.00 sec)
```

Justification

The column Name does not exist in the Student table. Replacing it with Student_Name creates the view successfully.

Q7. Examine an SQL query with unnecessary conditions and optimize it by simplifying the WHERE clause
(BL3 – Apply)

Create Table and Insert Data

```
mysql> CREATE TABLE Employee(  
-> Emp_ID INT PRIMARY KEY,  
-> Emp_Name VARCHAR(30),  
-> Salary INT  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> INSERT INTO Employee VALUES  
-> (101, 'Rahul', 30000),  
-> (102, 'Priya', 45000),  
-> (103, 'Kiran', 28000);  
Query OK, 3 rows affected (0.01 sec)  
Records: 3 Duplicates: 0 Warnings: 0
```

✗ Inefficient Query

```
mysql> SELECT *  
      -> FROM Employee  
      -> WHERE Salary>25000  
      -> AND Salary>20000  
      -> AND Salary>10000;
```

✓ Optimized Query

```
mysql> SELECT *  
      -> FROM Employee  
      -> WHERE Salary>25000;  
+-----+-----+-----+  
| Emp_ID | Emp_Name | Salary |  
+-----+-----+-----+  
|    101 | Rahul    | 30000 |  
|    102 | Priya    | 45000 |  
|    103 | Kiran    | 28000 |  
+-----+-----+-----+  
3 rows in set (0.00 sec)
```

Justification

The extra conditions are redundant because Salary > 25000 already satisfies the other two conditions. Removing unnecessary conditions improves readability and performance.

Q8. Identify violations of integrity constraints in a given table and correct them (BL2 – Understand)

Create Table and Insert Valid Data

```
mysql> CREATE TABLE Employee(  
    -> Emp_ID INT PRIMARY KEY,  
    -> Emp_Name VARCHAR(30) NOT NULL,  
    -> Email VARCHAR(50) UNIQUE,  
    -> Salary INT CHECK(Salary>0)  
    -> );  
Query OK, 0 rows affected (0.04 sec)  
  
mysql> INSERT INTO Employee VALUES  
    -> (101,'Rahul','rahul@gmail.com',35000);  
Query OK, 1 row affected (0.00 sec)  
  
mysql>  
mysql> INSERT INTO Employee VALUES  
    -> (102,'Priya','priya@gmail.com',42000);  
Query OK, 1 row affected (0.01 sec)
```

✗ Invalid Insert

```
mysql> INSERT INTO Employee VALUES  
    -> (103,NULL,'rahul@gmail.com',-5000);  
ERROR 1048 (23000): Column 'Emp_Name' cannot be null
```

✓ Correct Insert

```
mysql> INSERT INTO Employee VALUES  
    -> (103,'Kiran','kiran@gmail.com',30000);  
Query OK, 1 row affected (0.01 sec)
```

OUTPUT

```
mysql> SELECT * FROM Employee;
+-----+-----+-----+-----+
| Emp_ID | Emp_Name | Email          | Salary |
+-----+-----+-----+-----+
| 101    | Rahul    | rahul@gmail.com | 35000  |
| 102    | Priya    | priya@gmail.com | 42000  |
| 103    | Kiran    | kiran@gmail.com  | 30000  |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Justification

Integrity constraints ensure valid and consistent data by preventing null values, duplicate emails, and invalid salaries.

Q9. Fix a relation that violates 1NF or 2NF by removing repeating groups or partial dependencies
(BL3 – Apply)

Unnormalized Relation

Student_ID	Student_Name	Courses
101	Rahul	DBMS,Java
102	Priya	Python

Create Table and insert data

```
mysql> CREATE TABLE Student(
-> Student_ID INT PRIMARY KEY,
-> Student_Name VARCHAR(30)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Student_Course(
-> Student_ID INT,
-> Course_Name VARCHAR(30),
-> PRIMARY KEY(Student_ID,Course_Name),
-> FOREIGN KEY(Student_ID)
-> REFERENCES Student(Student_ID)
-> );
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> INSERT INTO Student VALUES
    -> (101,'Rahul'),
    -> (102,'Priya');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Student_Course VALUES
    -> (101,'DBMS'),
    -> (101,'Java'),
    -> (102,'Python');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

OUTPUT

```
mysql> SELECT * FROM Student;
+-----+-----+
| Student_ID | Student_Name |
+-----+-----+
|          101 | Rahul        |
|          102 | Priya        |
+-----+-----+
2 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Student_Course;
+-----+-----+
| Student_ID | Course_Name |
+-----+-----+
|          101 | DBMS        |
|          101 | Java        |
|          102 | Python      |
+-----+-----+
3 rows in set (0.00 sec)
```

Justification

Splitting the repeating group into a separate table satisfies 1NF, removes redundancy, and supports multiple courses for each student.

**Q10. Identify redundant attributes in a relation and apply normalization (up to 3NF)
(BL3 – Apply)**

Original Relation

Student_ID	Student_Name	Dept_Name	HOD
------------	--------------	-----------	-----

Create Table and Insert data

```
mysql> CREATE TABLE Department(  
-> Dept_ID VARCHAR(5) PRIMARY KEY,  
-> Dept_Name VARCHAR(30),  
-> HOD VARCHAR(30)  
-> );  
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> CREATE TABLE Student(  
-> Student_ID INT PRIMARY KEY,  
-> Student_Name VARCHAR(30),  
-> Dept_ID VARCHAR(5),  
-> FOREIGN KEY(Dept_ID)  
-> REFERENCES Department(Dept_ID)  
-> );  
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> INSERT INTO Department VALUES  
-> ('D101', 'CSE', 'Dr.Ramesh'),  
-> ('D102', 'ECE', 'Dr.Suresh');  
Query OK, 2 rows affected (0.01 sec)  
Records: 2 Duplicates: 0 Warnings: 0
```

```
mysql>  
mysql> INSERT INTO Student VALUES  
-> (101, 'Rahul', 'D101'),  
-> (102, 'Priya', 'D102'),  
-> (103, 'Kiran', 'D101');  
Query OK, 3 rows affected (0.01 sec)  
Records: 3 Duplicates: 0 Warnings: 0
```

Output

```
mysql> SELECT S.Student_ID,  
-> S.Student_Name,  
-> D.Dept_Name,  
-> D.HOD  
-> FROM Student S  
-> JOIN Department D  
-> ON S.Dept_ID=D.Dept_ID;
```

Student_ID	Student_Name	Dept_Name	HOD
101	Rahul	CSE	Dr.Ramesh
103	Kiran	CSE	Dr.Ramesh
102	Priya	ECE	Dr.Suresh

3 rows in set (0.00 sec)

Justification

Separating department information into a separate table removes redundant department and HOD details from the Student table. This satisfies 3NF, improves consistency, and eliminates update anomalies.